

Dicionários e conjuntos: organizar e cruzar dados

Atividade 1 · Consultar e atualizar por chave

Questões de autoavaliação

1. Registrar o estado final e distinguir atualização, adição de chave e remoção com pop.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

Original: 16; Não existe; {'Ana': 16, 'Rui': 14, 'Eva': 18}; True False. Após acrescentar Leo e remover Rui: {'Ana': 16, 'Eva': 18, 'Leo': 10}; o valor devolvido por pop é 14.

```
notas = {'Ana': 16, 'Rui': 12}
notas['Rui'] = 14
notas['Eva'] = 18
notas['Leo'] = 10
removida = notas.pop('Rui')
print(removida)
print(notas)
```

Uma atribuição a chave existente substitui o valor; a uma chave nova acrescenta um par. pop remove o par e devolve o valor associado.

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Atividade 2 · Contar ocorrências

Questões de autoavaliação

1. Registrar contagens antes e depois da normalização e explicar a distinção entre Ana e ana como chaves.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

Original: ana 3, rui 2, eva 1. Com 'Ana ana ANA Rui', sem normalizar: Ana 1, ana 1, ANA 1, Rui 1. Após lower: ana 3, rui 1.

```
texto = 'Ana ana ANA Rui'
frequencias = {}
for palavra in texto.lower().split():
    frequencias[palavra] = frequencias.get(palavra, 0) + 1
for palavra, quantidade in frequencias.items():
    print(palavra, quantidade)
```

Strings com maiúsculas diferentes são chaves distintas. Normalizar antes da contagem permite tratá-las como a mesma palavra.

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Dicionários e conjuntos: organizar e cruzar dados

Atividade 3 · Cruzar inscrições

Questões de autoavaliação

1. Identificar as mudanças nos conjuntos e explicar por que adicionar um elemento existente não aumenta o tamanho.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

Original: ambas [102]; só Python [101, 103]; pelo menos uma [101, 102, 103, 104]. Após adicionar 104: ambas [102, 104]; só Python [101, 103]; união mantém os quatro números. `len(python)` é 4.

Adicionar 102 novamente não muda o conjunto nem o seu tamanho. Conjuntos guardam elementos únicos. `sorted` apenas organiza a apresentação; um `set` não garante ordem de apresentação.

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Atividade 4 · Guardar dados para outra execução

Questões de autoavaliação

1. Comparar dados antes e depois da persistência. Explicar `wb`, `rb` e a relevância da origem do ficheiro.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

Após a alteração, alunos contém 101: Ana, 102: Rui, 103: Eva; inscritos contém 101 e 103. Os dados recuperados mantêm os mesmos pares e elementos. A ordem do `set` não é relevante.

`wb` abre em escrita binária e `rb` em leitura binária. `pickle` serializa objetos em bytes e recupera-os. A leitura de `pickle` pode executar código, pelo que só se devem usar dados de origem confiável, como o ficheiro criado na própria atividade.

```
# Acrescentar antes do bloco que guarda:  
dados['alunos'][103] = 'Eva'  
dados['inscritos'].add(103)
```

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Dicionários e conjuntos: organizar e cruzar dados

Atividade 5 · Aplicação integrada

Questões de autoavaliação

1. Explicar a organização do algoritmo e as decisões tomadas.
2. Comparar resultados previstos e observados em todos os testes pedidos.
3. Identificar uma falha possível e indicar como um teste a poderia revelar.

Resolução e verificação

```
def inscrever(alunos, inscritos, numero):  
    if numero not in alunos or numero in inscritos:  
        return False  
    inscritos.add(numero)  
    return True
```

```
alunos = {101: 'Ana', 102: 'Rui', 103: 'Eva'}  
inscritos = {101}  
print(inscrever(alunos, inscritos, 102))  
print(inscrever(alunos, inscritos, 102))  
print(inscrever(alunos, inscritos, 999))  
for numero in sorted(inscritos):  
    print(numero, alunos[numero])
```

As chamadas devolvem True, False e False. Saída dos inscritos: 101 Ana e 102 Rui. O dicionário mantém os três alunos; o conjunto final é {101, 102}. O teste de pertença ao dicionário impede inscrições de números desconhecidos.

Dicionários e conjuntos: organizar e cruzar dados

Atividade 6 · Identificadores duplicados

Questões de autoavaliação

1. Qual nome fica associado a 101 na versão inicial?
2. Qual o dicionário e conjunto de duplicados após a correção?
3. Por que um conjunto é adequado para registrar números repetidos sem os repetir no relatório?

Resolução e verificação

```
registos = [(101, 'Ana'), (102, 'Rui'), (101, 'Eva')]
alunos = {}
duplicados = set()
for numero, nome in registos:
    if numero in alunos:
        duplicados.add(numero)
    else:
        alunos[numero] = nome
print(alunos)
print(sorted(duplicados))
```

Inicial: 101 fica associado a Eva. Corrigido: {101:Ana,102:Rui}, duplicados {101}. Lista vazia produz {} e set(). Três ocorrências de 101 conservam o primeiro nome e registam 101 uma só vez nos duplicados.

Dicionários e conjuntos: organizar e cruzar dados

Atividade 7 · Auditar inscrições inconsistentes

Questões de autoavaliação

1. Quais são os desconhecidos e os inscritos em ambas?
2. Que operação identifica alunos sem qualquer inscrição?
3. Por que não se deve consultar alunos[numero] antes de excluir os desconhecidos?

Resolução e verificação

```
alunos = {101: 'Ana', 102: 'Rui', 103: 'Eva'}  
python = {101, 102, 999}  
bd = {102, 103, 888}  
conhecidos = set(alunos)  
desconhecidos = (python | bd) - conhecidos  
python_limpo = python & conhecidos  
bd_limpo = bd & conhecidos  
ambas = python_limpo & bd_limpo  
sem_inscricao = conhecidos - (python_limpo | bd_limpo)  
print('Desconhecidos:', sorted(desconhecidos))  
print('Ambas:', sorted(ambas))  
print('Sem inscrição:', sorted(sem_inscricao))  
for numero in sorted(ambas):  
    print(numero, alunos[numero])
```

Desconhecidos {888,999}; Python limpo {101,102}; BD limpo {102,103}; ambas {102}, nome Rui; sem inscrição set(). Com ambos vazios, desconhecidos e ambas ficam vazios e sem inscrição é {101,102,103}. Consulta direta de 999 ou 888 causaria KeyError.

Dicionários e conjuntos: organizar e cruzar dados

Atividade 8 · Atualizar inscrições com consistência

Questões de autoavaliação

1. Quais os estados após transferir 102 e repetir a tentativa?
2. Que erro ocorreria se se removesse da origem antes de validar todas as condições?
3. Como demonstrar com testes que uma rejeição conserva ambos os conjuntos?

Resolução e verificação

```
def transferir(alunos, origem, destino, numero):
    if numero not in alunos:
        return False
    if numero not in origem or numero in destino:
        return False
    origem.remove(numero)
    destino.add(numero)
    return True

alunos = {101: 'Ana', 102: 'Rui', 103: 'Eva'}
origem = {101, 102}
destino = {103}
assert transferir(alunos, origem, destino, 102)
assert origem == {101}
assert destino == {102, 103}
for numero in [102, 999, 103]:
    antes_o = origem.copy()
    antes_d = destino.copy()
    assert not transferir(alunos, origem, destino, numero)
    assert origem == antes_o and destino == antes_d
origem.add(103)
antes_o = origem.copy()
antes_d = destino.copy()
assert not transferir(alunos, origem, destino, 103)
assert origem == antes_o and destino == antes_d
print('Testes concluídos')
```

102 transfere com True: origem {101}, destino {102,103}. Repetir devolve False e conserva o estado; 999 e 103 também falham. Ao colocar 103 na origem, fica presente em ambos: a transferência continua rejeitada sem mudanças. Remover prematuramente poderia perder a inscrição ou lançar KeyError. Validar antes de alterar mantém a consistência.

Critério de autoavaliação: 2 pontos quando a implementação cumpre as regras, passa os testes pedidos e as respostas explicam os resultados; 1 ponto se falta parte da solução, dos testes ou da justificação; 0 pontos se a tarefa permanece por resolver. Aplicar o mesmo critério às restantes atividades, num total de 16 pontos. A pontuação orienta o estudo e não constitui classificação formal.